Appunti - Ingegneria del Software

Esame del 2025/11/03

Domanda 1.

> La variabile `x` rappresenta lo stato presente del sistema, la variabile `x1` rappresenta lo stato precedente ad 'x' e la variabile `x2` lo stato precedente ad `x1`.
> Si consideri il seguente requisito: il valore assoluto della differenza tra il valore dello stato presente e di quello precedente è al più 2.
> Quale delle seguenti `assert` definisce questo requisito (cioè ritorna errore quando il requisito non è verificato)?

- *Valore assoluto della differenza*: Si traduce matematicamente in $|x - x_1|$. In linguaggio di programmazione, questo è solitamente `abs(x - x1)`.
- *Al più 2*: In termini matematici, "al più" significa "minore o uguale a" (≤ 2).Quindi, la condizione che deve essere vera affinché il sistema funzioni correttamente è:

$$|x - x_1| \leq 2$$

Come funziona la `assert`
L'istruzione `assert(condizione)` funziona in questo modo:

- Se la condizione è **vera**, il programma prosegue normalmente.
- Se la condizione è **falsa**, la `assert` "fallisce" e ritorna un errore.
- Di conseguenza, dobbiamo inserire nella `assert` la condizione che definisce il comportamento corretto (il requisito). Se tale condizione non viene rispettata, l'`assert` genererà l'errore richiesto.

Quindi `assert( abs(x - x1) <= 2)` è ***corretta***. Verifica esattamente che lo scostamento tra lo stato presente x e quello precedente x_1 non superi il valore di 2.

Domanda 2.

> La variabile `x` rappresenta lo stato presente del sistema, la variabile `x1` rappresenta lo stato precedente ad `x` e la variabile `x2` lo stato precedente ad `x1`.
> Si consideri il seguente requisito: se lo stato vale 0 allora dopo al più 2 transizioni lo stato prende il valore 1. Quale delle seguenti `assert` definisce questo requisito (cioè ritorna errore quando il requisito non è verificato)?

Per verificare questo requisito guardando al passato (usando x, x_1 e x_2), dobbiamo invertire la prospettiva temporale: se lo stato due posizioni fa (x_2) era 0, allora lo stato precedente (x_1) oppure lo stato attuale (x) deve essere 1.

Scomposizione della logica `(condizione) ? (se vero) : (se falso)`

L'`assert` utilizza il *ternary operator*.

- Vediamo perché la `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1)` è ***corretta***:
  1. **Condizione** `(x2 == 0)`: Controlliamo se due passi fa il sistema era nello stato 0.
  2. **Se vero** `(x1 == 1) || (x == 1)`: Se due passi fa era 0, allora il requisito è soddisfatto solo se lo stato successivo (x_1) o quello attuale (x) è diventato 1.
  3. **Se falso** `: 1`: Se due passi fa lo stato **non** era 0, il requisito non è applicabile. In logica, una condizione "Se A allora B" è automaticamente vera se A è falso. Restituendo `1` (true), l'`assert` non segnala errori e il programma prosegue.

Domanda 3.

> Si consideri la seguente architettura software:
>
> "qimage.jpg" non trovato.
>
> Quale frammenti di psedo codice meglio la rappresenta?

Ogni riga dello pseudocodice segue la sintassi:

> `Input [etichetta] of [destinazione] = output [etichetta] of [sorgente]`.

Questo significa che nel diagramma si avrà una freccia con nome [etichetta] che parte da [sorgente] e va verso [destinazione].

Per esempio la riga `Input 12 of SC2 = output 12 of SC1` significa che ci deve essere una freccia con etichetta "12" che va dalla sorgente `SC1` alla destinazione `SC2`.
Per risolvere l'esercizio basta trovare il diagramma in cui ogni freccia corrisponde a una riga dello pseudocodice.

- La soluzione ***corretta*** di questo esercizio è la seguente:

```
Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 41 of SC1 = Output 41 of SC4
```

Domanda 4.

> La variabile `x` rappresenta lo stato presente del sistema, la variabile `x1` rappresenta lo stato precedente ad `x` e la variabile `x2` lo stato precedente ad `x1`.
> Si consideri il seguente requisito: la differenza tra il valore dello stato presente e di quello precedente è compresa tra -1 ed 1.
> Quale delle seguenti `assert` definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- La risposta ***corretta*** è: `assert((-1 <= (x - x1)) && (( x - x1) <= 1))`

Domanda 5.

> Un `Path` in un programma è una sequenza di istruzioni dall'inizio alla fine del programma. La `Path Coverage` di un insieme di `test cases` è la percentuale di `path` del programma che sono attraversati da almeno un test case.
> Considerando la seguente funzione `C`, qual'è la `path coverage` con i `tests cases` dati.
>
> "qimage (1).jpg" non trovato.
>
> `Test 1: x[0] = -1, x[1] = 1, x[2] = 5.`
> `Test 2: x[0] = 5, x[1] = 1, x[2] = 2.`
> `Test 3: x[0] = 3, x[1] = 3, x[2] = 0.`
> `Test 4: x[0] = 2, x[1] = 3, x[2] = -4.`

Questo esercizio si divide in *due parti*: trovare il numero totale di `path` e verificare quanti `path` vengono attraversati dai test.

I `path` sono semplicemente i rami dell'albero di `if-else`, quindi per contarli dobbiamo controllare le ramificazioni possibili.

- *1° `path`*: Se il primo `if` è vero controlliamo il secondo. Se il secondo `if` è vero ritorniamo 0.

- *2° path*: Se il primo `if` è vero e il secondo è falso, ritorniamo 1
- *3° path*: Se il primo `if` è falso, controlliamo l'`else-if`. Se è vero controlliamo il 3 `if`, che se anch'esso è vero ritorniamo 2.
- *4° path*: Se l'`else-if` è vero e il terzo `if` è falso ritorniamo 3
- *5° path*: Se il primo `if` è falso e l'`else-if` è falso, entriamo nell'`else` e ritorniamo 4.
  Quindi in totale ci sono 5 `path` possibili.

Per verificare quanti `path` vengono attraversati dai test, per ogni test sostituiamo gli input nel corpo della funzione

Test 1: $x = [-1, 1, 5]$

1. **Prima condizione:** $(-1)(-1) + (0)(1) + (4)(5) = 1 + 0 + 20 = 21$.
   - $21 \leq -4$? **FALSO**. Si passa al primo `else if`.
2. **Seconda condizione (`else if`):** $(0)(-1) + (-1)(1) + (-3)(5) = 0 - 1 - 15 = -16$.
   - $-16 \leq -4$? **VERO**. Entriamo nel blocco.
3. **Terza condizione (annidata):** $(3)(-1) + (0)(1) + (1)(5) = -3 + 0 + 5 = 2$.
   - $2 \leq 3$? **VERO**.

- **Risultato:** Ritorna **2**.
- `Path`: Condizione 1 (F) → Condizione 3 (V) → Condizione 4 (V).

Test 2: $x = [5, 1, 2]$

1. **Prima condizione:** $(-1)(5) + (0)(1) + (4)(2) = -5 + 0 + 8 = 3$.
   - $3 \leq -4$? **FALSO**. Si passa al primo `else if`.
2. **Seconda condizione (`else if`):** $(0)(5) + (-1)(1) + (-3)(2) = 0 - 1 - 6 = -7$.
   - $-7 \leq -4$? **VERO**. Entriamo nel blocco.
3. **Terza condizione (annidata):** $(3)(5) + (0)(1) + (1)(2) = 15 + 0 + 2 = 17$.
   - $17 \leq 3$? **FALSO**. Si va all' `else`.

- **Risultato:** Ritorna **3**.
- `Path`: Condizione 1 (F) → Condizione 3 (V) → Condizione 4 (F).

Test 3: $x = [3, 3, 0]$

1. **Prima condizione:** $(-1)(3) + (0)(3) + (4)(0) = -3 + 0 + 0 = -3$.
   - $-3 \leq -4$? **FALSO**. Si passa al primo `else if`.
2. **Seconda condizione (`else if`):** $(0)(3) + (-1)(3) + (-3)(0) = 0 - 3 - 0 = -3$.
   - $-3 \leq -4$? **FALSO**. Si va all'ultimo `else`.

- **Risultato:** Ritorna **4**.
- `Path`: Condizione 1 (F) → Condizione 3 (F).

Test 4: $x = [2, 3, -4]$

1. **Prima condizione:** $(-1)(2) + (0)(3) + (4)(-4) = -2 + 0 - 16 = -18$.
   - $-18 \leq -4$? **VERO**. Entriamo nel primo blocco `if`.
2. **Seconda condizione (annidata):** $(-1)(2) + (-4)(3) + (-4)(-4) = -2 - 12 + 16 = 2$.
   - $2 \leq -4$? **FALSO**. Si va all' `else`.

- **Risultato:** Ritorna **1**.
- `Path`: Condizione 1 (V) → Condizione 2 (F).
- **Path Coverage:** $4/5$ (80%) ossia lo ***0.800***. Manca solo il test case che soddisfi la condizione per ritornare **0** (ovvero quando sia la prima che la seconda condizione annidata sono vere).

Domanda 6.

> La `transition coverage` di un insieme di `test cases` (cioè sequenze di inputs) per uno `state diagram` è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta. Si consideri lo state `diagram UML` in figura:
>
> "qimage (2).jpg" non trovato.
>
> ed il seguente insieme di test cases:
>
> - Test case 1: act1 act1 act0 act2 act2
> - Test case 2: act2 act2 act1 act1
> - Test case 3: act2 act1
>
> Verificare quale è la stima migliore della transition coverage per i test cases di cui sopra.

Lo ***state coverage*** è il numero di stati visitati dai tutti i test case fratto il numero totale di stati.
Il ***transition coverage*** è numero di archi attraversati fratto numero di archi totale.

Quindi, per risolvere l'esercizio bisogna contare tutti gli stati attraversati dai test case e metterli a rapporto con il numero di stati totale.

Procediamo con la risoluzione:
Assumiamo che il sistema parta dallo **stato iniziale 0** (evidenziato in azzurro).
Test case 1: act1 $\rightarrow$ **act1** $\rightarrow$ **act0** $\rightarrow$ **act2** $\rightarrow$ **act2**

1. **0** $\xrightarrow{act1}$ **1** (Transizione percorse: 1)
2. **1** $\xrightarrow{act1}$ **2** (Transizione percorse: 2)
3. **2** $\xrightarrow{act0}$ **6** (Transizione percorse: 3)
4. **6** $\xrightarrow{act2}$ **0** (Transizione percorse: 4)
5. **0** $\xrightarrow{act2}$ **2** (Transizione percorse: 5)

Test case 2: act2 $\rightarrow$ **act2** $\rightarrow$ **act1** $\rightarrow$ **act1**
6. **0** $\xrightarrow{act2}$ **2** (Già percorsa)
7. **2** $\xrightarrow{act2}$ **0** (Transizione percorse: 6)
8. **0** $\xrightarrow{act1}$ **1** (Già percorsa)
9. **1** $\xrightarrow{act1}$ **2** (Già percorsa)

Test case 3: act2 $\rightarrow$ **act1**
10. **0** $\xrightarrow{act2}$ **2** (Già percorsa)
11. **2** $\xrightarrow{act1}$ **7** (Transizione percorse: 7)

Il diagramma presenta **20 transizioni** totali:

- **Dallo stato 0:** act1 (verso 1), act2 (verso 2), act0 (verso 6)
- **Dallo stato 1:** act1 (verso 2), act2 (verso 0), act0 (verso 6)
- **Dallo stato 2:** act1 (verso 7), act0 (verso 6), act2 (verso 0)
- **Dallo stato 3:** act1 (verso 7), act2 (verso 0), act0 (verso 1)
- **Dallo stato 4:** act1 (verso 0), act2 (verso 0), act0 (verso 6)
- **Dallo stato 5:** act1 (verso 1), act2 (verso 2), act0 (verso 1)
- **Dallo stato 6:** act2 (verso 0)
- **Dallo stato 7:** act2 (verso 3)
  Quindi abbiamo che il ***transition coverage*** è di $\frac{7}{20} = 0.300$

Domanda 7.

> Si consideri il processo software in figura
>
> “ex (2).jpg” non trovato.
>
> Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 8 mesi sia almeno 0.5 ?

Il tempo richiesto per la prima fase è 2 mesi, 4 mesi per la seconda e 2 per la terza, quindi la probabilità che il processo duri esattamente 8 mesi è la *probabilità* che il processo segua il
ercorso diretto senza mai tornare indietro e senza che venga mai ripetuta una fase.

La probabilità delle singole transizioni è: p (dalla 1 alle 2), 0.8 (dalla 2 alla 3) e 0.7 (dalla 3 alla 4)

L'evento, di cui stiamo cercando la probabilità, è l'intersezione degli eventi corrispondenti alle singole transizioni, e ciascuna transizione è un evento indipendente (proprietà di Markov), perciò possiamo semplicemente applicare il principio moltiplicativo:
$Ptot = p * 0.8 * 0.7 = p * 0.56$

L'esercizio ci chiede che la probabilità che il processo termini in 8 mesi sia almeno 0.5, perciò:
$Ptot >= 0.5$ e quindi: $p >= 0.5/0.56 = 0,893$

Domanda 8.

> Un processo software può essere rappresentato con un Markov Decision Process, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).
>
> A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.
>
> Si consideri il processo software in figura.
>
> “qimage (3).jpg” non trovato.
>
> Qual'è la migliore stima per il valore atteso del numero di iterazioni della "***SW Dev Phase 1***"?

Il *valore atteso* del numero di iterazioni di una qualsiasi fase in una Catena di Markov non è altro che il valore atteso del numero di fallimenti di una serie di prove bernoulliane indipendenti tra loro.

Parliamo quindi di una **variabile aleatoria geometrica**, e di conseguenza possiamo usare la formula del valore atteso di una variabile geometrica, che è $1/p$, dove p è la probabilità di successo. In questo caso la probabilità di successo è la probabilità di non ripetere la fase
(1 - probabilità del loop).

Quindi abbiamo: $1/(1 - 0.62) = 1/0,38 = 2,631$
Questo però *NON* è il risultato che stiamo cercando, perché quello che ci chiede il testo non è il valore atteso del numero di iterazioni necessarie a un successo, ma il *valore atteso del numero di fallimenti*. Per trovare questo valore possiamo semplicemente sottrarre 1 al valore atteso che abbiamo calcolato, ottenendo $1,631$.

Se chiede ad esempio, "per uscire", non si necessita di fare -1

Domanda 9.

> Si consideri la seguente funzione di transizione:
>
> "qimage (4).jpg" non trovato.
>
> Quale dei seguenti *UML state diagram* rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Analisi della Funzione di Transizione

La funzione definisce i seguenti passaggi chiave:

- **Dallo Stato 0:**
  - Se $u = 0 \implies$ ritorna **3**.
  - Se $u = 1 \implies$ ritorna **0**.
  - Se $u = 2 \implies$ ritorna **2**.
- **Dallo Stato 1:**
  - Se $u = 0 \implies$ ritorna **1**.
  - Se $u = 1 \implies$ ritorna **1**.
  - Se $u = 2 \implies$ ritorna **2**.
- **Dallo Stato 2:**
  - Se $u = 0 \implies$ ritorna **1**.
  - Se $u = 1 \implies$ ritorna **1**.
  - Se $u = 2 \implies$ ritorna **4**.
- **Dallo Stato 3:**
  - Se $u = 0 \implies$ ritorna **4**.
  - Se $u = 1 \implies$ ritorna **3**.
  - Se $u = 2 \implies$ ritorna **0**.
- **Dallo Stato 4:**
  - Se $u = 0 \implies$ ritorna **3**.
  - Se $u = 1 \implies$ ritorna **2**.
  - Se $u = 2 \implies$ ritorna **0**.

Quindi la soluzione corretta è:

"a1image.jpg" non trovato.

Domanda 11.

> Un processo software può essere rappresentato con un Markov Decision Process, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).
> A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.
> Si consideri il processo software in figura.
>
> "qimage (5).jpg" non trovato.
>
> Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Abbiamo che ogni transizione ha costo 100, e il costo totale delle transizioni dirette dalla fase 0 alla fase 4 è di 400, qui nell'esercizio ci chiede di calcolare la probabilità che il processo termini attraversando un percorso diretto
Possiamo semplicemente applicare il principio moltiplicativo sulle transizioni:
$0.93 * 0.89 * 0.87 = 0,720099$

Catena di Markov, probabilità del costo (500)

Quello che dobbiamo calcolare è la probabilità che il processo finisca con esattamente una transizione da una fase a sé stessa, ovvero:

- Loop in fase 1 OR Loop in fase 2 OR Loop in fase 3
  Dove Loop in fase i è l'evento in cui la fase i esegue un singolo loop e tutte le altri fasi non hanno loop. Gli eventi sono quindi disgiunti e possiamo semplicemente sommare le probabilità di ciascun evento:

  $$P(C = 500) = P(\text{Loop in fase 1}) + P(\text{Loop in fase 2}) + P(\text{Loop in fase 3})$$

  Ossia:
  $P(C = 500) = (\text{somma delle probabilità del loop}) * (\text{prodotto della probabilità degli archi uscenti})$
  Notiamo che a sinistra della moltiplicazione abbiamo semplicemente la somma delle probabilità di ogni loop, e a destra abbiamo la probabilità del percorso diretto.

Catena di Markov, probabilità del costo (600)

"qimage (6).jpg" non trovato.

Ci chiede di aggiungere 2 transizioni al percorso diretto anziché una sola, p per raggiungere il costo di 600.

Dobbiamo considerare percorsi con esattamente una transizione che torna indietro esattamente di una fase oppure per corsi con esattamente 2 loop. Usando lo stesso procedimento dell'esercizio 7 otteniamo che la formula finale è:

$$\begin{aligned} P(\text{percorso diretto}) \times (&\text{la somma delle probabilità delle transizioni all'indietro} \\ &+ \text{la somma delle probabilità delle combinazioni di 2 loop}) \end{aligned}$$

In questo caso, le transizioni all'indietro (esclusa da 3 a 1) sono quelle da 3 a 2 e da 2 a 1, con probabilità rispettivamente 0.03 e 0.06, quindi:

- Somma *delle probabilità delle transizioni all'indietro* = $(0.03 * 0.89) + (0.06 * 0,93) = 0.0825$

Le *possibili combinazioni di 2 loop sono*:

- $P(loop1 * loop1) = 0.07 * 0.07 = 0.0049$
- $P(loop2 * loop2) = 0.05 * 0.05 = 0.0025$
- $P(loop3 * loop3) = 0.04 * 0.04 = 0.0016$
- $P(loop1 * loop2) = 0.07 * 0.05 = 0.0035$
- $P(loop1 * loop3) = 0.07 * 0.04 = 0.0028$
- $P(loop2 * loop3) = 0.05 * 0.04 = 0.0020$
  La somma totale è 0.0173

La *probabilità del percorso diretto* è $0.93 * 0.89 * 0.87 = 0,720099$
La somma tra le probabilità delle transizioni all'indietro e le combinazioni di 2 loop è $0.0173 + 0.0825 = 0.0998$
Quindi:

- $0.720099 * 0.0998 = \text{circa } 0.0718$

Domanda 16.

> Il rischio R può essere calcolato come $R = P * C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.
> Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = exp(-b * S)$, dove b è una opportuna costante note da dati storici aziendali.
> $b = 0.000106$
> $C = 488564.000000$
> $R = 349.000000$
> Trova il costo S per lo sviluppo del software in questione.

L'unica cosa importante da ricordare è che per rimuovere l'esponenziale da un'uguaglianza bisogna usare il logaritmo naturale, il resto sono solo passaggi algebrici.
Quindi andiamo a calcolare:

$$R = P * C \implies P = R/C = 349/488564$$

$$P = exp(-b * S) \implies R/C = exp(-b * S) \implies ln(R/C) = -b * S$$

$$ln(R/C) = ln(349/488564) = -7.3$$

$$-7.3 = -(0.000106) * S \implies S = -7.3/-(0.000106) = \text{circa } 68000$$

Domanda 24.

La WBS (*Work Breakdown Structure*) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.
I vincoli di precedenza tra tasks sono dati nella tabella delle precedenze definita nel seguito.

"Screenshot 2026-02-07 183503.png" non trovato.

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla tabella degli efforts definita nel seguito:

"Screenshot 2026-02-07 183456.png" non trovato.

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.
Lo scheduling per i tasks è definito attraverso una tabella di scheduling . Ciascuna riga della tabella di scheduling consiste di una tripla(i, j, k) dove:

- i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
- j è il task
- k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0. Un esempio di tabella di scheduling è il seguente:

"Screenshot 2026-02-07 183740.png" non trovato.

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponendo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Per trovare la tabella giusta dobbiamo verificare 3 requisiti:

- *Vincolo di risorse*: in un singolo time slot non può esserci un effort maggiore al numero di personale
- *Vincolo di effort*: l'effort totale dedicato ad un task attraverso i time slot deve corrispondere a quello descritto nella tabella degli efforts
- *Vincolo di precedenza*: un task non può iniziare se un suo predecessore non è terminato.

La tabella corretta è di fatti:

"Screenshot 2026-02-07 184019.png" non trovato.

Domanda 26.

> Un processo software di n stati può essere modellato con una coppia (A, Y) dove:
>
> - A è una matrice che definisce una catena di Markov con n stati etichettati con interi da 0 ad $n-1$. Lo stato iniziale è etichettato con 0 mentre lo stato finale con $n-1$.
> - Y è un vettore di n elementi che definisce l'output della catena di Markov associando un output (se definito) a ciascuno stato.
> Quale delle seguenti rappresentazioni (A, Y) meglio modella il processo software in figura ?
>
> "ex (5).jpg" non trovato.

Gli elementi della matrice $A[i][j] = k$ indicano che la probabilità di passare dallo stato i allo stato j è k.
Gli elementi del vettore indicano l'output assegnato ad ogni stato. Per esempio dalla catena di Markov in figura abbiamo che $A[1][2] = 0.7$ e $Y[2] = 4$.

Per risolvere l'esercizio basta verificare quale risposta corrisponde alla catena data.

Domanda 48.

> Un processo software consiste delle seguenti fasi:
> 1: Requirements definition,
> 2: System development,
> 3: System testing.
> 4: Delivery (fase finale)
>
> La probabilità di passare da una fase i (i= 1, 2, 3) alla fase successiva $(i+1)$ è sempre 0.7, mentre con probabilità 0.3 è necessario ripetere la fase in corso a causa di errori.
> Quale delle seguenti catene di Markov meglio modella il processo software di cui sopra ?

Per risolvere questo esercizio ci basta *verificare che le probabilità nella catena di Markov* (rappresentate dalle frecce) corrispondano a quelle descritte.
Per esempio in questo caso, per $i = (1, 2, 3)$ abbiamo 0.7 di probabilità di passare da una fase i alla fase $i+1$, e 0.3 di probabilità di ripetere la stessa fase. La risposta 2 e la risposta 3 sono sbagliate perché preso $i = 3$ nessuna delle due ha probabilità 0.7 di passare alla fase 4, quindi la risposta giusta è la 2.

Esercizio

> Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: `F1` seguita da `F2`. Ciascuna fase ha costo 1000. Con probabilità 0.5 potrebbe essere necessario ripetere `F1` una seconda volta. Con probabilità 0.2 potrebbe essere necessario ripetere `F2` una seconda volta. Qual e' il *costo atteso* dello sviluppo dell'intero software?

Per calcolare il costo atteso di un processo iterativo, dobbiamo sommare i costi fissi delle fasi al valore atteso delle loro possibili ripetizioni.

$$
\begin{aligned}
E[C] &= \text{Costo}(F_1) + P(\text{ripeti } F_1) \cdot \text{Costo}(F_1) \\
&\quad + \text{Costo}(F_2) + P(\text{ripeti } F_2) \cdot \text{Costo}(F_2) \\
&= 1000 + (0.5 \times 1000) + 1000 + (0.2 \times 1000) \\
&= 1500 + 1200 = 2700
\end{aligned}
$$

Esercizio

> Per testare un sistema si vuole costruire un generatore che ogni T secondi invia un valore v a sistema da testare. Per ogni invio, il valore T è un valore intero scelto uniformemente a random nell'intervallo $[20, 30]$ mentre il valore v è un valore scelto aggiungendo, uniformemente a random, il valore -1 o $+1$ al valore precedente di v. Il valore iniziale di v è 0.
> Quale dei seguenti programmi meglio definisce il generator di cui sopra?

Per risolvere questo tipo di esercizi dovete vedere:

- Come è definito l'intervallo del valore T
- Come è aggiornato il valore v

La funzione `rand()%` n genera un valore casuale tra 0 e n -1. Questo vuol dire che per generare un intervallo da 20 a 30 inclusi, dobbiamo fare `20 + rand()% 11` (genera un valore casuale tra 0 e 10)
Per aggiornare v dobbiamo sommare randomicamente a v +1 o -1, ciascuno con probabilità del 50%. Questo effetto è ottenuto facendo `v = v -1+2*rand()%2` (genera 0 o 1).

Esercizio

> Un processo di sviluppo agile consiste di varie iterazioni. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione e sia A il costo di una iterazione. Il valore atteso del costo per l'iterazione è:

Poniamo E come *valore atteso* del costo per l'iterazione.
Sappiamo che l'iterazione verrà eseguita almeno una volta con costo A, e potrebbe essere ripetuta successivamente con probabilità p e costo A. Quindi:

- $E = A + p * A$
- $E = A * (p + 1)$

Spiegazione della formula utilizzata: Poniamo C come variabile aleatoria rappresentante il costo, e N come variabile aleatoria bernoulliana rappresentante il numero di iterazioni.
Abbiamo quindi che:

- $C = A + (N * A)$
  Cioè, il valore è del costo è una costante A per una singola iterazione, sommato ad A moltiplicato per il numero di altre iterazioni effettuate.

Il valore atteso del costo è quindi:

- $E[A + (N * A)]$
- Usando la proprietà di linearità del valore atteso questo diventa:
- $E[A] + E[(N * A)]$ = $A + A * E[N]$
- Il valore atteso di una variabile bernoulliana con probabilità p è sempre p, quindi:
- $A + A * E[N] = A + A * p$